PROCÉDURE COMPLÈTE HAUTE DISPONIBILITÉ (Heartbeat + DRBD + Apache)

PROCÉDURE COMPLÈTE HAUTE DISPONIBILITÉ (Heartbeat + DRBD + Apache)..... 1
Description de la mission :..... 2
Objectif:..... 2
1. PRÉPARATION DES SERVEURS..... 2
Tableau d'adressage..... 2
Installation (sur les 2 serveurs)..... 3
2. SCHÉMA RÉSEAU ET EXPLICATION..... 3
3. CONFIGURATION RÉSEAU..... 3
srv1..... 4
srv2..... 4
4. AJOUT ET PRÉPARATION DU DISQUE DRBD..... 4
5. CONFIGURATION DRBD..... 5
6. INITIALISATION DRBD..... 5
Passage en PRIMARY (srv1 uniquement)..... 6
7. FORMATAGE ET MONTAGE..... 6
8. LIEN AVEC APACHE..... 6
9. TEST DRBD (VALIDATION)..... 6
10. CONFIGURATION HEARTBEAT..... 7
ha.cf..... 7
haresources..... 8
authkeys..... 8
11. DÉMARRAGE..... 8
TESTS..... 8
Test 1 : Accès Web..... 9
Test 2 : IP virtuelle..... 9
Test 3 : Réplication DRBD..... 9
Test 4 : Failover..... 10
Test 5 : Après bascule..... 10
Test 6 : Failback..... 10
Test 7 : Robustesse..... 10
CONCLUSION..... 11

Description de la mission :

Description de la mission

Dans le cadre de cette mission, j'ai été chargé de mettre en place une infrastructure de haute disponibilité permettant d'assurer la continuité d'un service web en cas de panne.

L'objectif principal était de garantir qu'un site hébergé sur un serveur reste accessible même si celui-ci devient indisponible. Pour cela, j'ai déployé une solution reposant sur deux serveurs Linux configurés en cluster.

La solution mise en œuvre s'appuie sur :

- DRBD, qui permet la réplication en temps réel des données entre les deux serveurs
- Heartbeat, qui gère la supervision des serveurs et la bascule automatique (failover)
- Apache, utilisé pour héberger le service web

Le fonctionnement est le suivant : un serveur est actif (PRIMARY) et héberge le service, tandis que le second reste en attente (SECONDARY). En cas de défaillance du serveur principal, le serveur secondaire prend automatiquement le relais, récupère l'adresse IP virtuelle et assure la continuité du service sans interruption pour les utilisateurs.

Cette mission m'a permis de comprendre les enjeux de la haute disponibilité, notamment en termes de tolérance aux pannes, de réplication des données et de gestion automatique des services critiques.

Objectif:

Mettre en place une infrastructure haute disponibilité avec :

- Réplication des données (DRBD)
- Bascule automatique (Heartbeat)
- Service web Apache hautement disponible

---

1. PRÉPARATION DES SERVEURS

Tableau d'adressage

srv1	Srv2	ip virtuelle
10.158.45.80	10.158.45.79	10.158.45.100

Installation (sur les 2 serveurs)

sudo apt update
sudo apt install apache2 drbd-utils heartbeat net-tools openssh-server -y

---

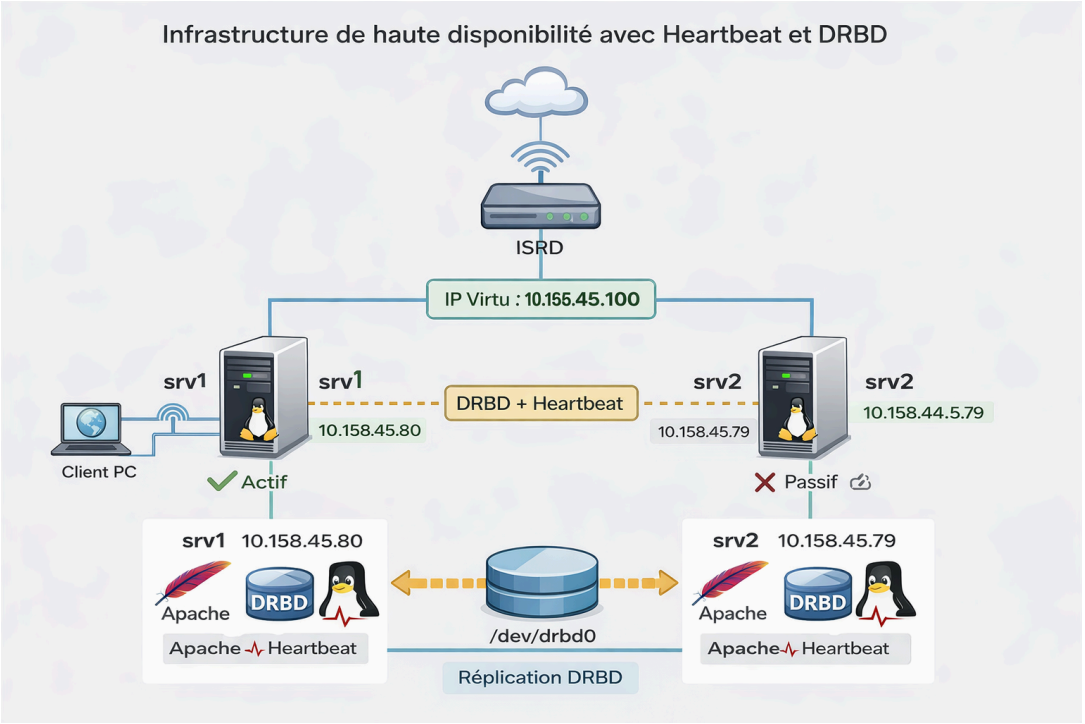
2. SCHÉMA RÉSEAU ET EXPLICATION

Architecture :

- 2 serveurs en cluster
- Communication DRBD entre srv1 et srv2
- IP virtuelle gérée par Heartbeat

Fonctionnement :

- srv1 = serveur principal (PRIMARY)
- srv2 = serveur secondaire (SECONDARY)
- En cas de panne → bascule automatique vers srv2



---

3. CONFIGURATION RÉSEAU

srv1

sudo nano /etc/netplan/01-netcfg.yaml

```
network:
  version: 2
  ethernets:
    enp0s3:
      addresses:
        - 10.158.45.80/24
      dhcp4: no
```

srv2

sudo nano /etc/netplan/01-netcfg.yaml

```
network:
  version: 2
  ethernets:
    enp0s3:
      addresses:
        - 10.158.45.79/24
      dhcp4: no
```

sudo netplan apply

---

4. AJOUT ET PRÉPARATION DU DISQUE DRBD

Objectif : utiliser un disque secondaire pour la réplication.

Sur les 2 serveurs :

lsblk
sudo fdisk /dev/sdb

Dans fdisk :

n
p
Entrée
Entrée
w

Résultat attendu : création de /dev/sdb1

---

5. CONFIGURATION DRBD

sudo nano /etc/drbd.d/r0.res

```
resource r0 {
  protocol C;

  on srv1 {
    device /dev/drbd0;
    disk /dev/sdb1;
    address 10.158.45.80:7789;
    meta-disk internal;
  }

  on srv2 {
    device /dev/drbd0;
    disk /dev/sdb1;
    address 10.158.45.79:7789;
    meta-disk internal;
  }
}
```

---

6. INITIALISATION DRBD

Sur les 2 :

```
sudo drbdadm create-md r0
sudo drbdadm up r0
```

```
user@srv1:~$ sudo drbdadm create-md r0
[sudo] password for user:
You want me to create a v08 style flexible-size internal meta data block.
There appears to be a v08 flexible-size internal meta data block
already in place on /dev/sdb1 at byte offset 5367656448

Do you really want to overwrite the existing meta-data?
[need to type 'yes' to confirm] yes

md_offset 5367656448
al_offset 5367623680
bm_offset 5367459840

Found ext3 filesystem
     5241660 kB data area apparently used
     5241660 kB left usable by current configuration

Even though it looks like this would place the new meta data into
unused space, you still need to confirm, as this is only a guess.

Do you want to proceed?
[need to type 'yes' to confirm] yes

initializing activity log
initializing bitmap (160 KB) to all zero
Writing meta data...
New drbd meta data block successfully created.
user@srv1:~$
```

Passage en PRIMARY (srv1 uniquement)

sudo drbdadm primary --force r0

---

7. FORMATAGE ET MONTAGE

Sur srv1 :

sudo mkfs.ext4 /dev/drbd0
sudo mkdir /mnt/sharefs
sudo mount /dev/drbd0 /mnt/sharefs

---

8. LIEN AVEC APACHE

sudo systemctl stop apache2
sudo rm -rf /var/www/html
sudo ln -s /mnt/sharefs /var/www/html
sudo systemctl start apache2

---

9. TEST DRBD (VALIDATION)

Sur srv1 :

echo "Serveur 1 actif" | sudo tee /mnt/sharefs/index.html

Sur srv2 :

sudo mkdir /mnt/sharefs
sudo mount /dev/drbd0 /mnt/sharefs
cat /mnt/sharefs/index.html

Résultat attendu :
"Serveur 1 actif"

```
user@srv2:~$ echo "Serveur 1 actif" | sudo tee /mnt/sharefs.index.html
[sudo] password for user:
Serveur 1 actif
user@srv2:~$
```

---

10. CONFIGURATION HEARTBEAT

ha.cf

sudo nano /etc/ha.d/ha.cf

logfile /var/log/ha-log
keepalive 2
deadtime 10
warntime 5
initdead 20
udpport 694

ucast enp0s3 10.158.45.79

node srv1
node srv2

```
  GNU nano 7.2

logfile /var/log/ha-log
keepalive 2
deadtime 10
warntime 5
initdead 20
udpport 694

ucast enp0s3 10.158.45.80   # sur srv1
ucast enp0s3 10.158.45.99_  # sur srv2

node srv1
node srv2
```

haresources

sudo nano /etc/ha.d/haresources

srv1 IPaddr2::10.158.45.100/24/enp0s3 apache2

```
  GNU nano 7.2
srv1 IPaddr2::10.158.45.100/24/enp0s3 apache2
```

authkeys

sudo nano /etc/ha.d/authkeys

auth 1
1 crc

sudo chmod 600 /etc/ha.d/authkeys

---

11. DÉMARRAGE

sudo systemctl enable heartbeat
sudo systemctl start heartbeat

TESTS

Test 1 : Accès Web

curl 10.158.45.100

Résultat : serveur 1 actif

```
root@srv2:~# curl 10.158.45.80
Serveur 1 actif
root@srv2:~# _
```

Test 2 : IP virtuelle

ip a

✔ visible uniquement sur le serveur actif

```
user@srv1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fc:3f:16 brd ff:ff:ff:ff:ff:ff
    inet 10.158.45.80/24 brd 10.158.45.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fefc:3f16/64 scope link
```

```
user@srv2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ba:48:d5 brd ff:ff:ff:ff:ff:ff
    inet 10.158.45.79/24 brd 10.158.45.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feba:48d5/64 scope link
       valid_lft forever preferred_lft forever
user@srv2:~$ ping 10.158.45.80
PING 10.158.45.80 (10.158.45.80) 56(84) bytes of data.
64 bytes from 10.158.45.80: icmp_seq=1 ttl=64 time=1.33 ms
64 bytes from 10.158.45.80: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 10.158.45.80: icmp_seq=3 ttl=64 time=1.89 ms
```

Test 3 : Réplication DRBD

echo "TEST" | sudo tee -a /mnt/sharefs/index.html

```
user@srv1:~$ sudo -i
root@srv1:~# echo "MODIF TEST" | tee -a /mnt/sharefs/index.html
MODIF TEST
root@srv1:~# _
```

Sur srv2 :

cat /mnt/sharefs/index.html

✔ contenu identique

Test 4 : Failover

sudo shutdown now

✔ srv2 devient actif
✔ IP bascule (10.158.45.100/24)
✔ site accessible

```
root@srv2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ba:48:d5 brd ff:ff:ff:ff:ff:ff
    inet 10.158.45.79/24 brd 10.158.45.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet 10.158.45.100/24 brd 10.158.45.255 scope global secondary enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feba:48d5/64 scope link
       valid_lft forever preferred_lft forever
root@srv2:~#
```

Test 5 : Après bascule

curl 10.158.45.100

✔ site fonctionnel

Test 6 : Failback

sudo reboot
cat /proc/drbd

✔ UpToDate/UpToDate

Test 7 : Robustesse

Répéter plusieurs fois :

sudo shutdown now

✔ toujours accessible
✔ aucune perte de données

CONCLUSION

Cette architecture garantit :

- Haute disponibilité
- Tolérance aux pannes
- Réplication temps réel des données